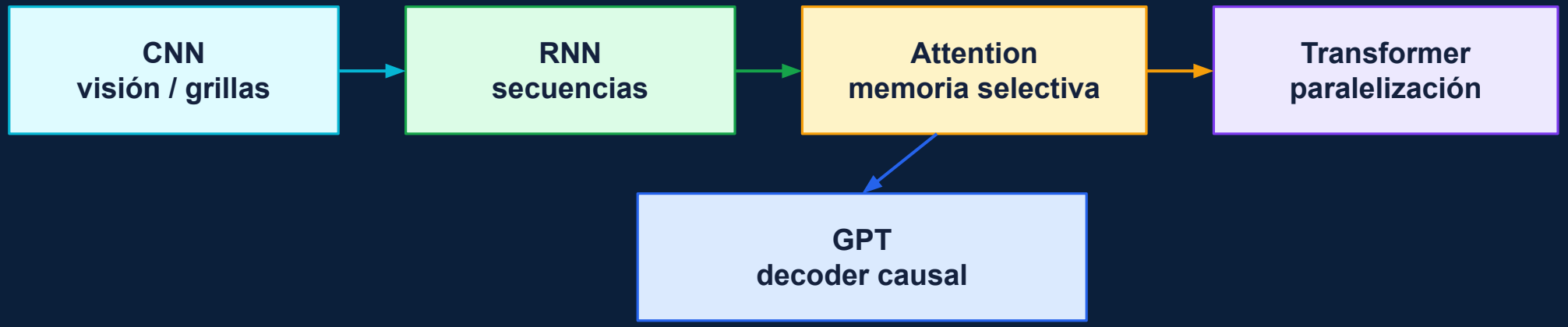
Redes convolucionales, RNN, Attention, Transformers y GPT

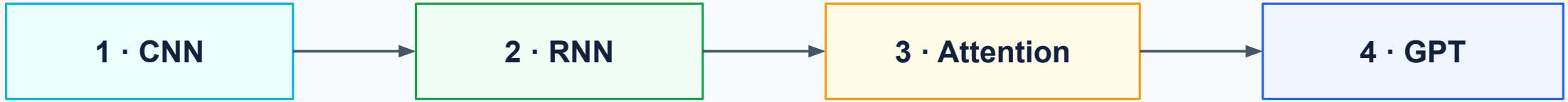
Clase 28 de mayo de 2026 · Inteligencia Artificial · 4to año de Lic. en Sistemas de Información

Profesores: Mg. Díaz Santiago R. · AS. López Joaquín · BioIng. Hadad Alejandro



Objetivo de la clase

Entender cómo las arquitecturas modernas explotan estructura: espacio, tiempo y contexto.



CNN: convolución, filtros, padding, stride, pooling y backpropagation.
RNN: recurrencia, memoria temporal, BPTT, LSTM/GRU y estabilidad del gradiente.
Attention/Transformer: embeddings, atención escalada, multi-head, máscaras causales.
Cierre: arquitectura GPT, entrenamiento autoregresivo y análisis de error.

Notación mínima

La clase usa álgebra lineal, cálculo diferencial y probabilidades.

$$x \in \mathbb{R}^{H \times W \times C}, \quad y \in \{1, \dots, K\}, \quad \theta = \{W, b\}$$

$$\hat{y} = f_\theta(x), \qquad \mathcal{L}(\theta) = \frac{1}{n} \sum_{i=1}^{n} \ell(f_\theta(x_i), y_i)$$

Idea transversal: una red transforma representaciones. Cada capa aprende una función diferenciable para minimizar una pérdida.

En deep learning no diseñamos a mano todos los atributos: aprendemos filtros, estados, embeddings y pesos de atención.

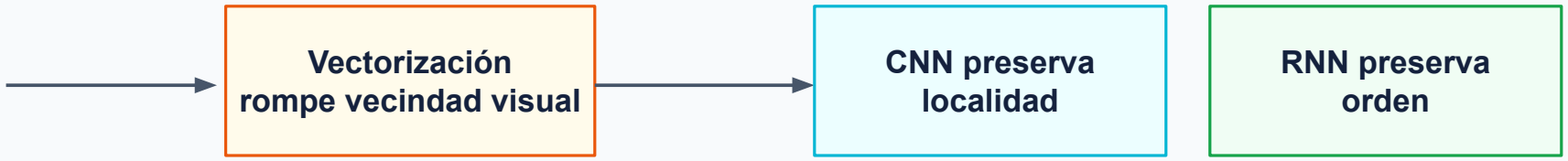
Espacio: píxeles vecinos son relevantes.
Tiempo: el orden de la secuencia importa.
Contexto: una palabra depende de otras palabras.

Por qué no basta con una red densa

Una capa densa ignora estructura local y escala mal con entradas grandes.

$$z = Wx + b, \quad W \in \mathbb{R}^{m \times d}$$

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	16



Principio de diseño: imponer una hipótesis inductiva adecuada reduce parámetros y mejora generalización.

BLOQUE I

Redes Convolucionales

Matemática de filtros, mapas de características, backpropagation y ejemplos.

Clase 28 de mayo · Inteligencia Artificial · 4to año

Convolución discreta 2D

En CNN se implementa usualmente correlación cruzada, pero se la llama convolución.

$$Y[i,j] = \sum_{u=0}^{k_h-1} \sum_{v=0}^{k_w-1} K[u,v]\, X[i+u, j+v]$$

$$(K * X)[i,j] = \sum_{u} \sum_{v} K[u,v] X[i-u, j-v]$$

Aprender un kernel K equivale a aprender detectores locales: bordes, texturas, esquinas, patrones o piezas de objetos.

Ejemplo de filtro vertical

1	0	-1
1	0	-1
1	0	-1

Localidad: el filtro mira una ventana pequeña.
Compartición de pesos: el mismo filtro se desplaza por toda la imagen.
Equivarianza traslacional: mover la entrada mueve el mapa de activación.

Ejemplo numérico: un filtro sobre una imagen

Una salida es un producto interno local entre parche y kernel.

Entrada X

1	2	0
0	1	3
2	1	1

Kernel K

1	0
0	-1

Salida Y

0	**-1**
-1	**0**

$$Y[0,0] = 1 \cdot 1 + 0 \cdot 2 + 0 \cdot 0 + (-1) \cdot 1 = 0$$

$$Y[0,1] = 1 \cdot 2 + 0 \cdot 0 + 0 \cdot 1 + (-1) \cdot 3 = -1$$

Atención: según orientación del kernel y definición exacta, los signos pueden cambiar. En librerías se usa correlación cruzada y la red aprende los pesos.

Cálculo completo usando correlación cruzada válida: [0, -1; -1, 0]. Si se usa otro kernel, el signo cambia. Lo importante es la operación local diferenciable.

Canales y tensores

Una imagen RGB y cada capa intermedia son tensores con canales.

$$X \in \mathbb{R}^{H \times W \times C_{in}}, \quad K \in \mathbb{R}^{k_h \times k_w \times C_{in} \times C_{out}}$$

$$Y[:, :, c_o] = \sum_{c_i=1}^{C_{in}} K[:, :, c_i, c_o] \star X[:, :, c_i] + b_{c_o}$$



Cada filtro combina información de todos los canales de entrada y produce un canal de salida. El bias es uno por filtro/canal de salida.

Stride, padding y tamaño de salida

Controlan resolución, cobertura y pérdida de borde.

$$H_{out} = \left\lfloor \frac{H + 2P - k_h}{S} \right\rfloor + 1, \quad W_{out} = \left\lfloor \frac{W + 2P - k_w}{S} \right\rfloor + 1$$

Padding P
agrega borde

Stride S
salta posiciones

Kernel k
ventana local

Resolución
de salida

Padding “same”: intenta conservar H y W.
Stride mayor: reduce costo y resolución.
Kernel grande: ve más contexto, pero agrega parámetros y costo.
En visión moderna se combinan convoluciones y pooling/stride para jerarquías.

Por qué CNN escala mejor que una capa densa

La compartición de pesos reduce dimensionalidad efectiva.

$$\#\theta_{conv} = k_h k_w C_{in} C_{out} + C_{out}$$

$$\#\theta_{dense} = HWC_{in} \cdot M + M$$

Ejemplo: imagen 224×224×3, 64 filtros 3×3 → conv ≈ 1.792 parámetros; densa a 64 neuronas → ≈ 9.633.856 parámetros.

Menos parámetros no implica menor expresividad: la red apila filtros y construye jerarquías.
La hipótesis inductiva es que patrones locales pueden repetirse en diferentes posiciones.
Esto mejora generalización cuando el dato realmente tiene estructura espacial.

Bias en CNN: interpretación matemática

El bias desplaza el umbral de activación de cada filtro.

$$a_{i,j,c} = \sum_{u,v,c_i} K_{u,v,c_i,c}\, X_{i+u,j+v,c_i} + b_c$$

$$h_{i,j,c} = \phi(a_{i,j,c})$$

Sin bias, un filtro solo se activa si el producto interno supera el umbral impuesto por el origen. Con bias, la red aprende qué tan fácil o difícil debe ser activar ese detector.

Filtro: “borde vertical” → **b bajo**
se activa fácil

b alto
se activa solo con evidencia fuerte

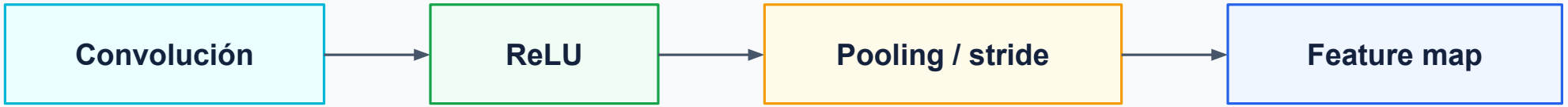
Ejemplo interpretativo: si b es negativo, puede exigir que el patrón local sea muy parecido al filtro para superar ReLU; si b es positivo, puede activar aun con evidencia parcial.

No linealidad y pooling

La convolución sola es lineal; la profundidad útil requiere no linealidades.

$$\mathrm{ReLU}(z) = \max(0, z), \quad \sigma(z) = \frac{1}{1 + e^{-z}}, \quad \tanh(z) = \frac{e^{z} - e^{-z}}{e^{z} + e^{-z}}$$

$$\mathrm{MaxPool}(X)_{i,j} = \max_{(u,v) \in \Omega_{i,j}} X_{u,v}$$



Pooling agrega robustez local, pero puede descartar información fina. Por eso muchas arquitecturas modernas usan stride o atención según el caso.

Forward propagation en una CNN

La red compone operadores diferenciables y produce logits.

$$H^{(\ell)} = \phi\left(K^{(\ell)} \star H^{(\ell-1)} + b^{(\ell)}\right)$$

$$\hat{p} = \mathrm{softmax}(W\,\mathrm{vec}(H^{(L)}) + b)$$



Los logits no son probabilidades: softmax los normaliza.
Para regresión se suele reemplazar softmax por salida lineal.
Para segmentación se predice una etiqueta por píxel, no una por imagen.

Backpropagation en convolución

Los gradientes también son sumas locales: la operación es diferenciable.

$$\frac{\partial \mathcal{L}}{\partial K[u,v]} = \sum_{i,j} \frac{\partial \mathcal{L}}{\partial Y[i,j]} \, X[i+u, j+v]$$

$$\frac{\partial \mathcal{L}}{\partial b} = \sum_{i,j} \frac{\partial \mathcal{L}}{\partial Y[i,j]}$$

$$K \leftarrow K - \eta \nabla_K \mathcal{L}$$

Interpretación: si un filtro produce activaciones que aumentan el error, el gradiente modifica sus pesos para activarse menos ante esos parches.

El aprendizaje de un filtro es una acumulación estadística sobre muchos parches.
La tasa de aprendizaje η controla el tamaño del paso.
Optimizadores como Adam reescalan el gradiente usando momentos.

Teorema: equivarianza por traslación

La convolución conmuta con traslaciones bajo condiciones de borde adecuadas.

$$T_{\Delta}(K \star X) = K \star T_{\Delta}(X)$$

Lectura: si el objeto se desplaza en la imagen, el mapa de activación se desplaza de manera equivalente. La red no necesita reaprender el mismo patrón en cada posición.

Esto no es invariancia total: la salida se mueve.
Pooling, agregación global y data augmentation ayudan a obtener invariancia aproximada.
La propiedad se rompe parcialmente por padding, bordes y muestreo discreto.



Pérdidas y métricas en CNN

La función de pérdida depende del tipo de salida.

$$\mathcal{L}_{CE} = -\sum_{k=1}^{K} y_k \log \hat{p}_k$$

$$\mathrm{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

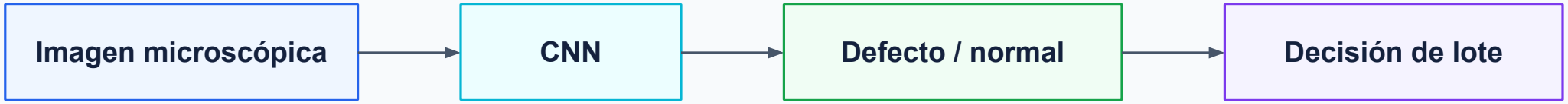
$$\mathrm{IoU} = \frac{|A \cap B|}{|A \cup B|}$$

Clasificación: cross-entropy + accuracy, F1, matriz de confusión.
Regresión: MSE, MAE, RMSE y análisis de residuales.
Segmentación: IoU/Dice y errores por clase o región.

Una métrica no reemplaza el análisis de error: mirar casos fallidos suele revelar sesgo de datos, fuga de información o clases mal definidas.

Ejemplo real poco usual: defectos en semillas por imagen microscópica

La CNN aprende patrones visuales sutiles que no son obvios para una regla manual.



Entrenamiento: imágenes de semillas históricas con etiqueta validada.
Validación: semillas de lotes no vistos para ajustar hiperparámetros.
Test: lotes de campaña nueva, nunca usados durante diseño.
Riesgo: si entreno y testeo con semillas del mismo lote, la red memoriza iluminación o cámara.

Criterio crítico: dividir por lote/origen, no al azar por imagen individual, para evitar fuga de información.

División de datos y validación cruzada

El objetivo no es ajustar el pasado, sino estimar desempeño fuera de muestra.

$$\hat{R}_{CV} = \frac{1}{K} \sum_{k=1}^{K} R\left(f_{\theta^{(-k)}}, D_k\right)$$



Holdout: simple, rápido, sensible a la partición.
K-fold: estima variabilidad, pero es caro en deep learning.
Group K-fold: útil cuando hay lotes, pacientes, clientes o sesiones.
Time split: obligatorio cuando el futuro no debe contaminar el pasado.

Underfitting, overfitting, sesgo y varianza

El análisis de error guía decisiones de datos, modelo y regularización.

$$\mathbb{E}\left[(y - \hat{f}(x))^2\right] = \mathbf{Bias}^2 + \mathbf{Varianza} + \sigma^2$$

Underfitting
alto sesgo

Buen ajuste
equilibrio

Overfitting
alta varianza

Alto train error + alto val error → modelo insuficiente o features pobres.
Bajo train error + alto val error → exceso de capacidad, fuga o pocos datos.
Error sistemático por subgrupo → sesgo de dataset o etiquetas.
Regularización: data augmentation, weight decay, dropout, early stopping.

BLOQUE II

Redes Recurrentes

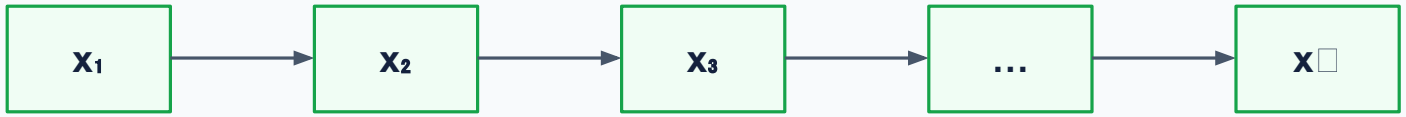
Secuencias, memoria temporal, BPTT, estabilidad y compuertas.

Clase 28 de mayo · Inteligencia Artificial · 4to año

Datos secuenciales

Una observación no es solo un vector: es una trayectoria ordenada.

$$x_{1:T} = (x_1, x_2, \ldots, x_T), \quad y_{1:T} \text{ o } y$$



Ejemplos: texto, audio, logs de sensores, series financieras, clicks, ADN.
La hipótesis temporal: el pasado contiene información sobre el presente o futuro.
RNN procesa la secuencia compartiendo los mismos pesos en cada tiempo.

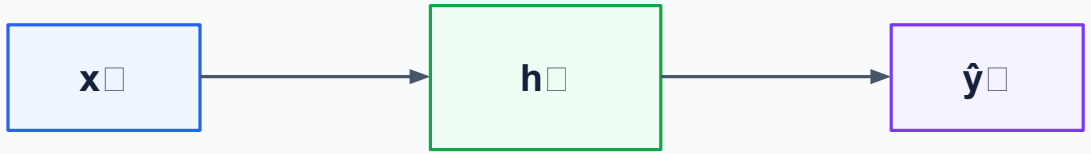
RNN vanilla: estado oculto recurrente

El estado h□ comprime información del pasado.

$$h_t = \phi(W_x x_t + W_h h_{t-1} + b_h)$$

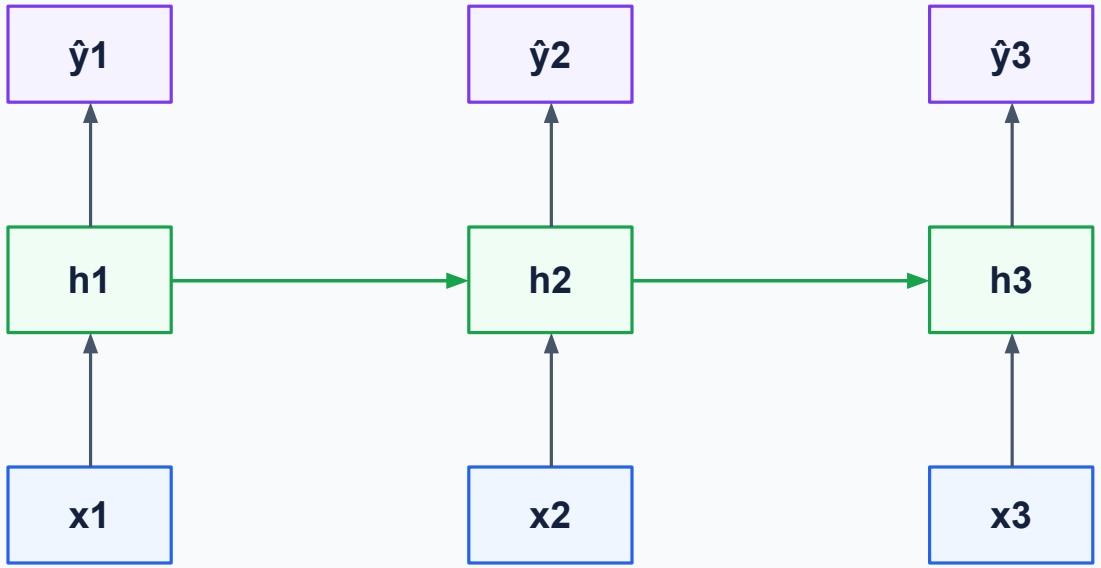
$$\hat{y}_t = g(W_y h_t + b_y)$$

El mismo W_x, W□ y W_y se reutiliza en todos los pasos. Esa compartición permite procesar secuencias de longitud variable.



RNN desenrollada

Entrenar una RNN equivale a entrenar una red profunda en el tiempo.



$$\mathcal{L} = \sum_{t=1}^{T} \ell(\hat{y}_t, y_t)$$

Muchos-a-uno: clasificación de una secuencia completa.
Muchos-a-muchos: etiquetado por tiempo, traducción, audio.
El gradiente se propaga hacia atrás por todos los pasos temporales.

Backpropagation Through Time

El gradiente multiplica jacobianos a lo largo de la secuencia.

$$\frac{\partial \mathcal{L}}{\partial W_h} = \sum_{t=1}^{T} \sum_{k=1}^{t} \frac{\partial \mathcal{L}_t}{\partial h_t} \left(\prod_{j=k+1}^{t} \frac{\partial h_j}{\partial h_{j-1}} \right) \frac{\partial h_k}{\partial W_h}$$

La multiplicación repetida puede hacer que el gradiente desaparezca o explote. Este es el problema central de RNN largas.

||J1 · J2 · ... · JT|| <= ||J1|| · ||J2|| · ... · ||JT||

Si las normas son < 1 → gradiente tiende a 0.
Si las normas son > 1 → gradiente puede explotar.
Soluciones: LSTM/GRU, clipping, normalización, atención.

Teorema práctico: memoria larga y espectro

La estabilidad depende del radio espectral de las transformaciones recurrentes.

$$h_t = \phi(W_h h_{t-1} + W_x x_t), \quad J_t = \mathrm{diag}(\phi'(a_t))W_h$$

$$\rho(J) < 1 \Rightarrow \|J^T\| \to 0, \qquad \rho(J) > 1 \Rightarrow \|J^T\| \text{ puede crecer}$$

Lectura: para recordar dependencias lejanas, la red necesita rutas de gradiente estables. LSTM crea “carriles” de memoria con compuertas.

ρ(J) es el radio espectral: magnitud máxima de autovalores.
En práctica la no linealidad y los datos hacen el análisis variable por tiempo.
Aun así, la intuición explica por qué RNN vanilla suele fallar con contextos largos.

LSTM: memoria con compuertas

La celda separa memoria c□ y estado h□.

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f), \quad i_t = \sigma(W_i[h_{t-1}, x_t] + b_i)$$

$$\tilde{c}_t = \tanh(W_c[h_{t-1}, x_t] + b_c), \quad c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$$

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o), \quad h_t = o_t \odot \tanh(c_t)$$

Forget f□
qué borrar

Input i□
qué escribir

Cell c□
memoria

Output o□
qué exponer

El bias de la compuerta de olvido suele inicializarse positivo para favorecer recordar al inicio del entrenamiento.

GRU: una alternativa más compacta

Combina memoria y estado con menos compuertas que LSTM.

$$z_t = \sigma\big(W_z[h_{t-1}, x_t]\big), \quad r_t = \sigma\big(W_r[h_{t-1}, x_t]\big)$$

$$\tilde{h}_t = \tanh\big(W_h[r_t \odot h_{t-1}, x_t]\big), \quad h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$$

z□ decide cuánto actualizar.
r□ decide cuánto pasado usar para el candidato.
Suele ser más rápida que LSTM y competitiva en secuencias medianas.
Para contextos muy largos, attention/transformers suelen dominar.

No hay una arquitectura universalmente superior: depende de longitud, tamaño de datos, ruido, costo y necesidad de interpretabilidad.

Pérdidas para secuencias

Se puede predecir en cada tiempo o una sola salida final.

$$\mathcal{L}_{seq} = \sum_{t=1}^{T} \ell(\hat{y}_t, y_t)$$

$$\mathcal{L}_{final} = \ell(\hat{y}_T, y)$$

many-to-one
texto → sentimiento

many-to-many
señal → etiqueta por paso

En series temporales, validación debe respetar cronología.
La matriz de confusión aplica si la salida es categórica.
MAE/MSE aplican si la salida es continua.
El error acumulativo importa en predicción autoregresiva.

Ejemplo real poco usual: detección de anomalías en bombas de riego

La señal cambia antes de la falla; la RNN aprende patrones temporales.



Train: campañas anteriores completas.
Validación: últimos meses antes de test, sin mezclar futuro.
Test: campaña nueva o equipos no vistos.
Error crítico: entrenar con ventanas superpuestas y repartirlas aleatoriamente contamina test.

Para secuencias, el split correcto suele ser por tiempo, por equipo o por evento; no por ventana individual.

Limitaciones de RNN

Procesar paso a paso reduce paralelismo y dificulta memoria larga.

Secuencial lento en GPU

Gradiente inestable

Contexto comprimido en h□

Difícil escalar

$$h_T = F(x_1, x_2, \ldots, x_T) \quad \text{como cuello de botella}$$

Attention cambia la pregunta: en lugar de comprimir todo en un estado, permite consultar directamente partes relevantes de la secuencia.

BLOQUE III

Attention y Transformer

Embeddings, atención escalada, multi-head, codificador/decodificador y máscaras.

Embeddings: convertir símbolos en geometría

Un token discreto se representa como un vector entrenable.

$$e_i = E^\top \text{onehot}(i), \quad E \in \mathbb{R}^{V \times d}$$



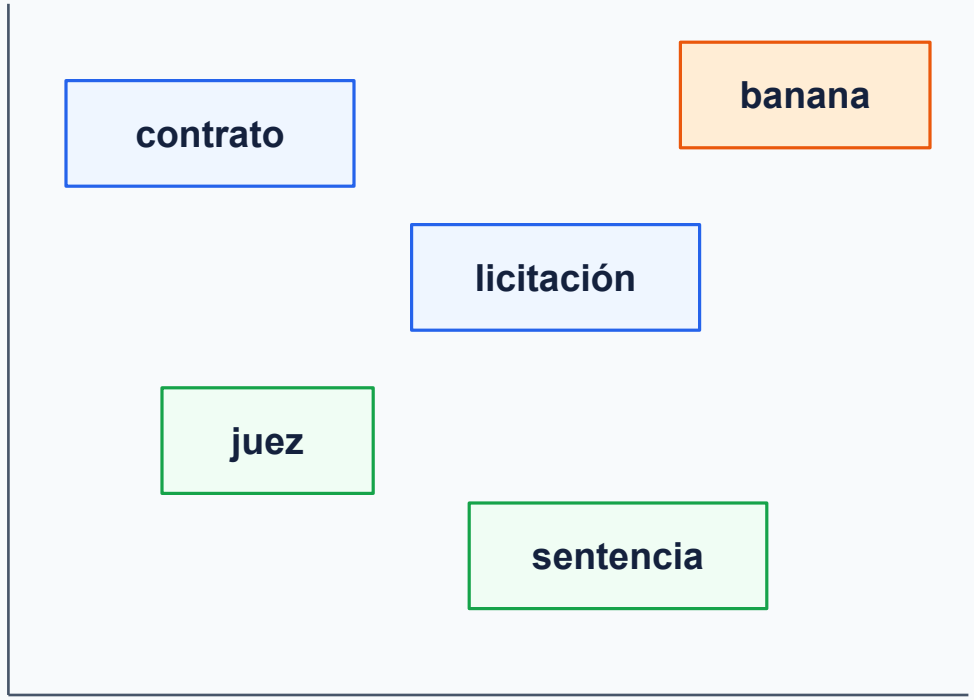
Semántica distribuida: significado como patrón de coordenadas.
Similitud: tokens con usos parecidos tienden a estar cerca.
El embedding se ajusta por gradiente junto con la red.
No “entiende” por magia: aprende regularidades estadísticas del corpus.

Geometría de embeddings

La similitud se mide con producto interno o coseno.

$$\cos(e_i, e_j) = \frac{e_i^\top e_j}{||e_i||\ ||e_j||}$$

$$P(w_t \mid contexto) \propto \exp(e_{w_t}^\top c)$$



El espacio aprendido no es único: puede rotarse o reescalarse sin cambiar ciertas relaciones internas del modelo.

Posición: el orden no viene gratis

Self-attention pura no distingue el orden sin codificación posicional.

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{2i/d}}\right), \quad PE(pos, 2i+1) = \cos\left(\frac{pos}{10000^{2i/d}}\right)$$

$$z_t = e_t + p_t$$

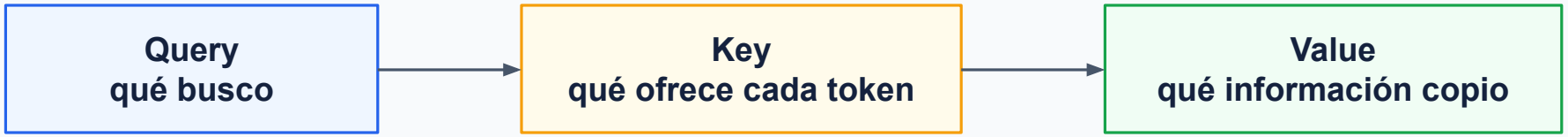


Codificación sinusoidal: fija y extrapolable.
Posiciones aprendidas: una tabla entrenable.
Modelos modernos usan variantes relativas o rotacionales según arquitectura.

Queries, Keys y Values

Attention es búsqueda diferenciable de información relevante.

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V$$



Si una query se parece a una key, ese token recibe mayor peso y su value influye más en la representación resultante.

Scaled Dot-Product Attention

La atención es una combinación convexa de values ponderada por compatibilidad.

$$\mathrm{Attention}(Q, K, V) = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$$

$$\alpha_{ij} = \frac{\exp(q_i^\top k_j / \sqrt{d_k})}{\sum_m \exp(q_i^\top k_m / \sqrt{d_k})}, \qquad z_i = \sum_j \alpha_{ij} v_j$$

La división por √d□ evita que los productos internos crezcan demasiado y saturen la softmax.



Multi-Head Attention

Varias cabezas permiten mirar relaciones distintas en paralelo.

$$\mathbf{head}_r = \mathbf{Attention}(XW_Q^{(r)}, XW_K^{(r)}, XW_V^{(r)})$$

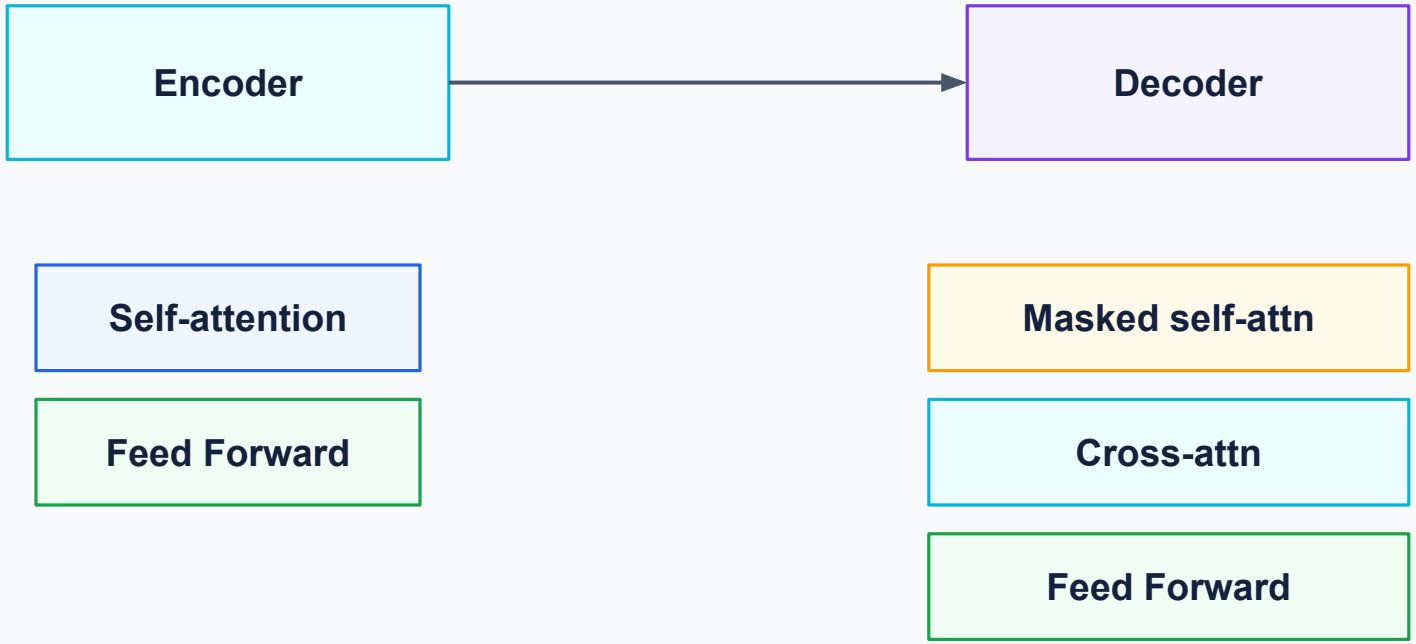
$$\mathrm{MHA}(X) = \mathrm{Concat}(head_1, \ldots, head_h)W_O$$



No se fuerza que cada cabeza tenga un rol humano interpretable, pero empíricamente muchas capturan relaciones diferentes.

“Attention Is All You Need”

El Transformer reemplaza recurrencia por atención paralelizable.



Ventaja central: los tokens pueden atenderse entre sí en paralelo, mejorando escalabilidad frente a RNN.

$$\text{Transformer} = \text{Attention} + \text{MLP} + \text{Residual} + \text{Norm}$$

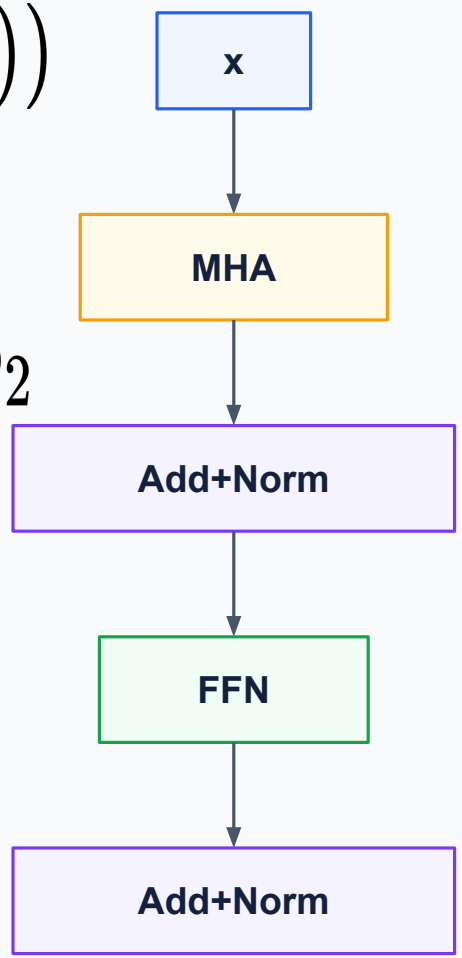
Bloque Transformer

Cada bloque mezcla información contextual y luego transforma cada posición.

$$u = \mathrm{LayerNorm}(x + \mathrm{MHA}(x))$$

$$y = \mathrm{LayerNorm}(u + \mathrm{FFN}(u))$$

$$\mathrm{FFN}(u) = W_2\phi(W_1u + b_1) + b_2$$



Residual conecta caminos cortos para el gradiente; LayerNorm estabiliza escalas de activación.

Máscara causal en decodificadores

Para generar texto, el token t no puede mirar el futuro.

$$M_{ij} = \begin{cases} 0, & j \leq i \\ -\infty, & j > i \end{cases}$$

$$\text{softmax}\left(\frac{QK^\top + M}{\sqrt{d_k}}\right)V$$

Matriz de máscara causal

0	-∞	-∞	-∞
0	0	-∞	-∞
0	0	0	-∞
0	0	0	0

Esto convierte al Transformer en un modelo autoregresivo: predice el próximo token usando solo el pasado.

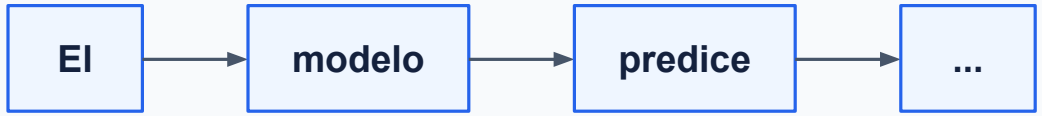
Objetivo autoregresivo

GPT aprende a predecir el próximo token.

$$P(x_{1:T}) = \prod_{t=1}^{T} P(x_t \mid x_{<t})$$

$$\mathcal{L}(\theta) = -\sum_{t=1}^{T} \log P_\theta(x_t \mid x_{<t})$$

En inferencia, se elige el siguiente token por greedy, sampling, temperature, top-k o nucleus sampling.



Costo computacional de attention

La matriz de atención crece cuadráticamente con la longitud.

$$QK^\top \in \mathbb{R}^{T\times T}, \quad \text{costo} \approx O(T^2 d)$$

$$\text{memoria} \approx O(T^2)$$

Ventaja: paralelismo alto en hardware moderno.
Costo: secuencias largas requieren memoria cuadrática.
Soluciones: sparse attention, sliding window, kernels eficientes, chunking.
Trade-off: contexto largo vs. latencia y costo.

Una arquitectura no se evalúa solo por accuracy: también por costo, latencia, consumo, robustez y facilidad de mantenimiento.

Interpretar attention con cuidado

Los pesos de atención sugieren relaciones, pero no son explicación causal completa.

$$Z = AV, \quad A = \mathrm{softmax}(QK^\top / \sqrt{d_k})$$

A: matriz de atención

0.70	0.10	0.20
0.15	0.75	0.10
0.25	0.25	0.50

Una fila indica cuánto consulta un token a otros tokens.
Atención alta no implica necesariamente causalidad.
Para análisis serio: ablation, contraejemplos, gradientes y perturbaciones.
En dominios sensibles, documentar errores por subgrupo y contexto.

Evaluación en clasificación, regresión y generación

La métrica debe corresponder al riesgo real del sistema.

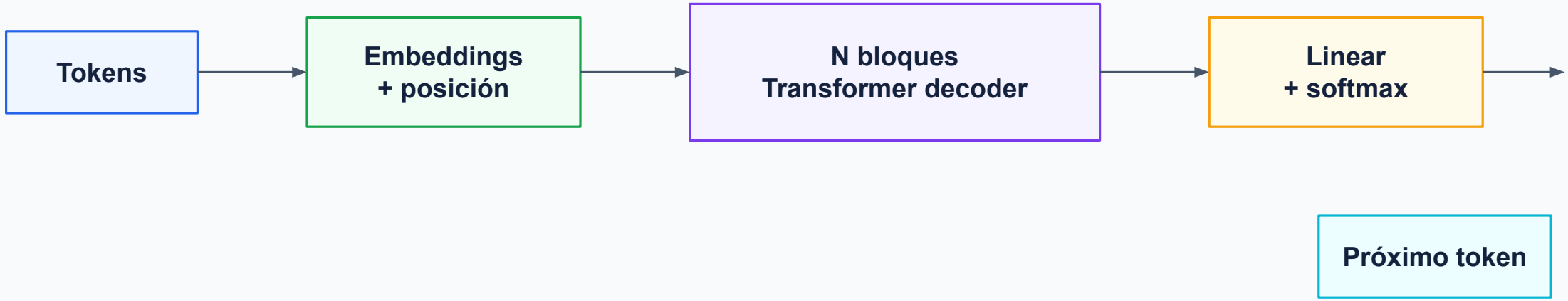
$$\mathbf{Perplexity} = \exp\left(-\frac{1}{T}\sum_{t=1}^{T}\log P(x_t \mid x_{<t})\right)$$

$$\mathbf{AUC} = \int_0^1 TPR(FPR)\,dFPR$$

Clasificación: matriz de confusión, precision, recall, F1, ROC-AUC.
Regresión: MSE/MAE y distribución de errores.
Generación: perplexity, evaluación humana, factualidad, robustez, seguridad.
No mezclar test con decisiones de diseño; usar validación separada.

Arquitectura GPT: Transformer decoder-only

GPT apila bloques de decodificador con máscara causal.



$$x_t \mapsto e_t + p_t \mapsto H_t^{(N)} \mapsto \mathrm{softmax}(W_{vocab} H_t^{(N)})$$

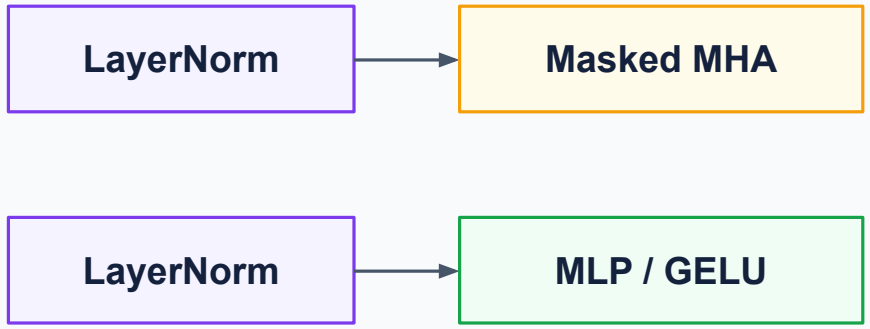
No hay encoder separado. La máscara causal impone que la predicción de t use solo $x_1, \ldots, x_{t-1}$.

Dentro de un bloque GPT

La información fluye por self-attention causal y una red feed-forward.

$$h \leftarrow h + \mathrm{MHA}_{causal}(\mathrm{LN}(h))$$

$$h \leftarrow h + \mathrm{MLP}(\mathrm{LN}(h))$$



Pre-LN suele estabilizar entrenamiento profundo.
Residuals preservan gradiente y señal original.
MLP expande dimensión interna y aplica no linealidad.
La salida se proyecta al vocabulario para obtener logits.

Ejemplo de flujo GPT: de texto a probabilidad

El modelo transforma un prompt en distribución sobre el vocabulario.



$$P(\text{“neuronal”} \mid \text{“La red”}) = \frac{e^{z_{neuronal}}}{\sum_{v \in V} e^{z_v}}$$

Sampling no es “pensar”: es elegir tokens según una distribución aprendida. La calidad depende de datos, entrenamiento, alineación y contexto.

Cierre: mapa conceptual final

CNN, RNN y Transformer son respuestas distintas a la estructura del dato.

CNN
localidad espacial

RNN
orden temporal

Attention
contexto selectivo

GPT
decoder causal

CNN aprende filtros compartidos y jerarquías visuales.
RNN procesa secuencias con estados y compuertas.
Transformer reemplaza recurrencia por atención paralela.
GPT modela texto como predicción autoregresiva del próximo token.

Referencias base: LeCun et al. (1998), Hochreiter & Schmidhuber (1997), Cho et al. (2014), Bahdanau et al. (2014), Vaswani et al. (2017), Radford et al. (2018–2019).